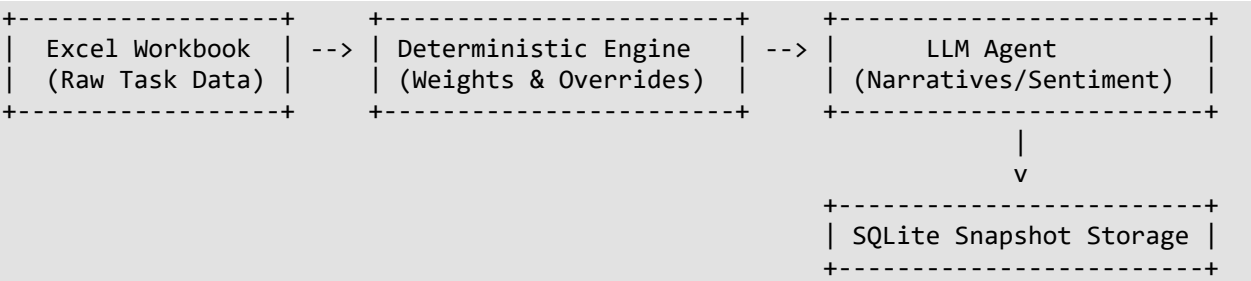


How Project Works

This document defines the methodology, features, technology stack, and data processing workflows for the **Project Health Reporting Agent**. It explains how the project works end-to-end, the systemic architecture, and the precise rule engine methodology.

1. RAG Determination Methodology (How the Engine Works)

The core philosophy of the system is **deterministic, auditable scoring**. Instead of delegating RAG classification to black-box LLMs—which are susceptible to hallucination, inconsistency, and lack mathematical validation—the RAG classification is computed via a structured rule engine. The LLM acts as an explanatory and summarization layer *on top* of the verified metrics.



1.1 Weighted Signals

Every project snapshot is evaluated across six foundational signal layers. Each signal yields a normalized score between 0 (lowest risk) and 100 (highest risk).

Signal	Weight	Computation Logic
Schedule Slippage	30%	Scans the schedule health column for explicit "Red", "Amber", or "Yellow" flags; checks total float erosion; and scores late active tasks.
Progress Gap	20%	Compares actual percent complete against the expected progress linearly calculated from the elapsed project timeline.
Milestone Health	20%	Analyzes incomplete key milestones, measuring the ratio of delayed or high-risk milestones to completed ones.
Blockers & Critical Path	15%	Identifies tasks on the critical path, negative total float, and checks task comments/predecessors for blocker keywords (e.g., <i>blocked</i> , <i>pending</i> , <i>delay</i>).
Stakeholder Sentiment	10%	Evaluates the emotional tone, outlook, and thematic urgency of PM comments on a scale from 0 to 100.
Budget Burn	5%	Calculates cost variance and burn rates. If budget metrics are missing from the plan, this signal is excluded, and weights are re-normalized.

1.2 Scoring Normalization Formula

The overall project health score ($S_{project}$) is calculated as: $S_{project} = \frac{\sum_i W_i \cdot S_i}{\sum_i W_i}$ Where W_i is the weight of signal i , and S_i is the normalized score of signal i . If a signal is unavailable (e.g., budget data is absent), its weight W_i is set to zero, and the remaining weights are re-normalized to sum to \$1.0\$.

1.3 RAG Status Mapping

The final score ($S_{project}$) maps to a status:

- *  **Green:** $0 \leq S_{project} < 35$ (No critical overrides triggered)
- *  **Amber:** $35 \leq S_{project} < 70$
- *  **Red:** $70 \leq S_{project} \leq 100$

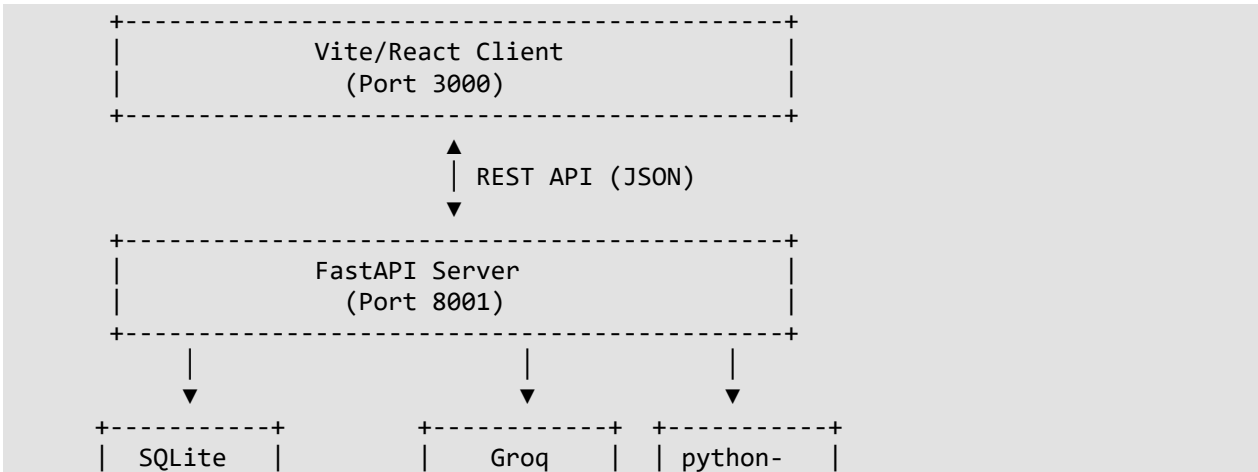
1.4 Critical Override Rules

To capture severe systemic risks that a weighted average might dilute, the engine applies **deterministic override rules** to force a project to **Red**:

1. **Severe Critical Path Delay:** An uncompleted critical path task delayed by more than 15 working days.
2. **Milestone Failure:** A key milestone that has missed its baseline finish date by more than 30 days and remains incomplete.
3. **Multiple Blockers:** More than 3 active tasks flag-marked as blocked by external dependencies.
4. **Widespread Red Schedule:** Over 40% of active tasks flagged as "Red" or "Yellow" schedule health in the workbook.

2. Technical Stack Methodology

The architecture utilizes a decoupled client-server model designed for performance, local deployability, and robust offline operations. The separation of concerns ensures that the AI reasoning layers are independent of the deterministic data parsing components.



Database	LLM API	pptx
+-----+	+-----+	+-----+

2.1 Backend Core

- * **Language:** Python 3.10+ (Static type annotations enforced). Python was chosen for its robust data manipulation libraries and AI integration capabilities.
- * **REST API:** FastAPI + Uvicorn (Bound to Port 8001 to bypass default port conflicts). Provides high-performance async endpoints for the frontend.
- * **Database:** SQLite (sqlite3 with row factory enabled) to persist project metadata, task attributes, RAG signals, comments, and quality metrics. Chosen for its zero-configuration local storage.
- * **AI reasoning:** Groq client accessing llama-3.3-70b-versatile (or fallback Google Gemini API / OpenAI API) via a structured JSON-schema tool-calling interface. This provides the intelligence layer for synthesizing project narratives.
- * **Excel Engine:** openpyxl with custom memory loading and explicit handle disposal to prevent file locks on Windows environments.
- * **Slide Engine:** python-pptx implementing automated layout coordinate grids and font color normalization to produce executive monthly presentations.

2.2 Frontend Client

- * **Framework:** React 19 + TypeScript + Vite. Provides a highly responsive user interface with strict type safety.
- * **Routing:** TanStack Router (Typesafe routes).
- * **Styling:** Tailwind CSS + Shadcn UI primitive blocks. Ensures a modern, accessible, and cohesive design system.
- * **Icons:** Lucide React.
- * **State Management:** React Query (TanStack Query) for declarative caching and optimistic mutations, managing data fetching from the FastAPI backend.

3. Product Features

The system features are designed to create a seamless pipeline from raw data to executive insight:

- * **Resilient Excel Ingestion:** Reads multi-sheet Excel files. Reconstructs hierarchies dynamically using indentation levels or Ancestors values, normalizing disparate date formats, null values, and custom fields.
- * **Active Stage Auto-Detection:** If a project stage is missing from the workbook summary, the system automatically walks the task hierarchy tree to determine the current, incomplete phase.
- * **Interactive RAG Explanations:** A dedicated dashboard tab ("Why this status") displays the reasoning behind the RAG status, listing the triggering signal details and override indicators.

- * **Audit Trails:** The "Signals" tab provides a breakdown of every computed signal (weights, raw values, normalized risk scores, and evidence).
 - * **Raw Data Preservation:** Under "Preserved Attributes", users can inspect every original column value parsed from the workbook, ensuring no information is discarded.
 - * **Project Switcher:** A header dropdown selector lets users switch between different projects within the portfolio, updating all tabs dynamically.
 - * **Cascade Snapshot Deletion:** Users can delete any uploaded snapshot. The system performs cascade deletions across SQLite tables, unlinks Markdown reports, and cleans up orphaned project directories.
 - * **Executive Slides Generation:** Compiles monthly summaries into a 16:9 widescreen presentation deck using the RAG logic and AI insights.
-

4. End-to-End Data Lifecycle (How It Works in Practice)

1. **File Upload:** The user uploads an `.xlsx` file via the frontend dropzone.
2. **Parsing & Normalization:** The backend reads the file, cleans cells, assigns unique row numbers, and saves the hierarchy.
3. **Signal Extraction:** The backend's deterministic engine computes metrics for slip, progress gap, milestones, blockers, and sentiment.
4. **AI Synthesis:** The computed signals and key task rows are packaged into a structured prompt. The LLM processes this context and returns narrative summaries explaining *why* the metrics look the way they do.
5. **Storage:** The workbook structure, raw data, computed signals, RAG status, and AI summaries are saved as a new snapshot in the SQLite database.
6. **Presentation:** The frontend queries the REST API to load snapshots, display the dashboard, or trigger PowerPoint slide compilation for monthly reporting.